Paw Tracker

Dog Sport Tracker App
Sprint3

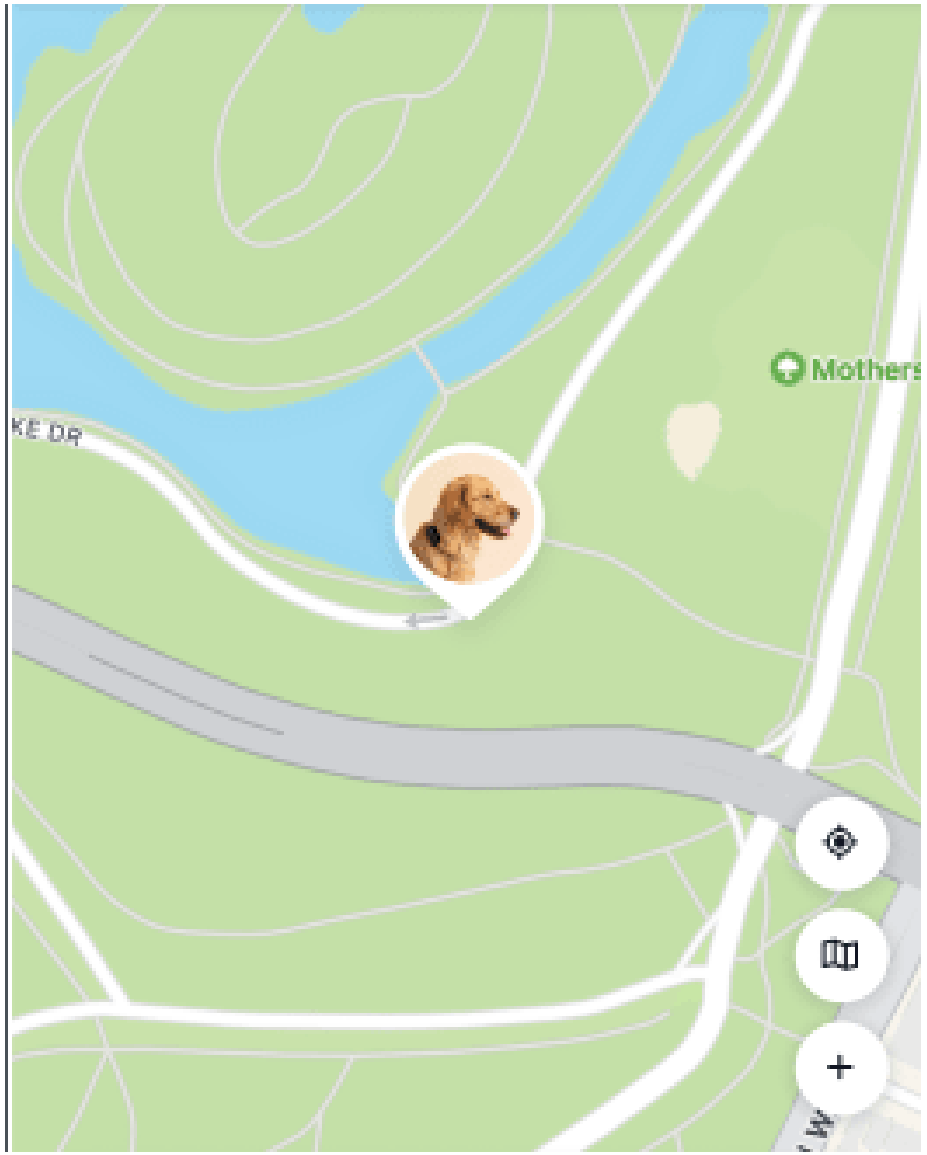
Mobile Application Development Project

Farnaz Jamilpanah (Srum Master)

Emmi Korhonen

Sprint 3 Goals:

- Fix daily and weekly filtering logic in HistoryScreen
- Fix distance and duration format
- Implement daily progress chart
- Update Figma
- Integrate daily activity goals in ProfileScreen
- Add polyline string to database
- Save goals to Database
- Unit test, UI test
- Calculate sum today’s/week's walks

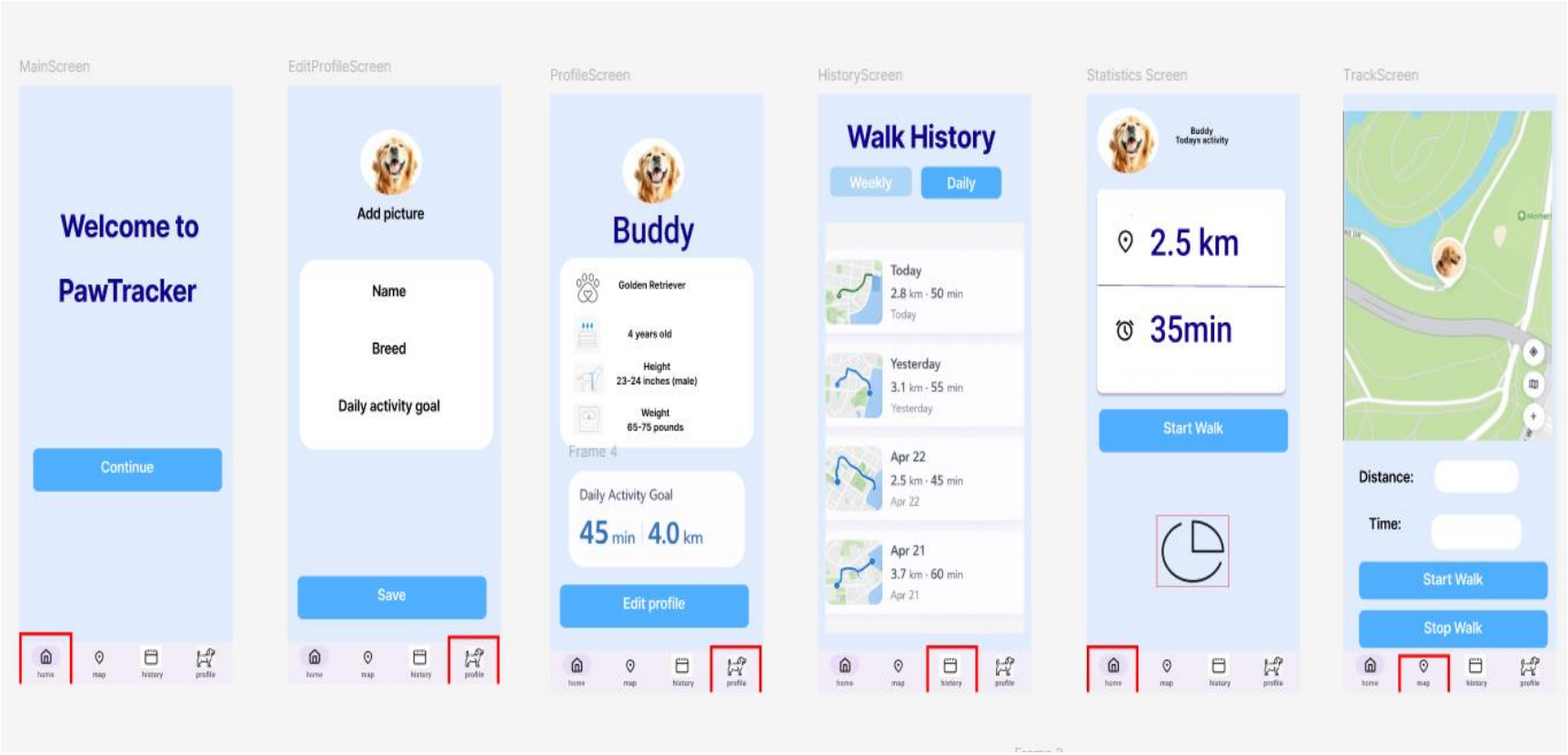


Jira:

Sprint 3 8 Apr – 15 Apr (14 work items)

APP-66	As a user, I want to see a live map of my walk route so that I can recognize the runs by area	DONE
APP-62	As a user, I want to set daily walk goals so I can stay consistent	DONE
APP-86	As a user, I want to see my daily walking statistics, so I can understand my activity for today.	DONE
APP-87	As a user, I want to visualize my progress.	DONE
APP-88	Fix daily and weekly UI filter logic (HistoryScreen)	DONE
APP-89	Fix distance and duration format	DONE
APP-90	Chart Implementation(StatisticsScreen)	DONE
APP-91	Integrate daily activity goal and edit button(ProfileScreen)	DONE
APP-92	Add polyline string to database	DONE
APP-93	Update gps points during tracking(trackingViewModel)	DONE
APP-94	Save goals to database	DONE
APP-95	Sum today/week's walks(totalDistanceKm, totalDurationMs)	DONE
APP-96	Unit test	TO DO
APP-97	UI test	TO DO

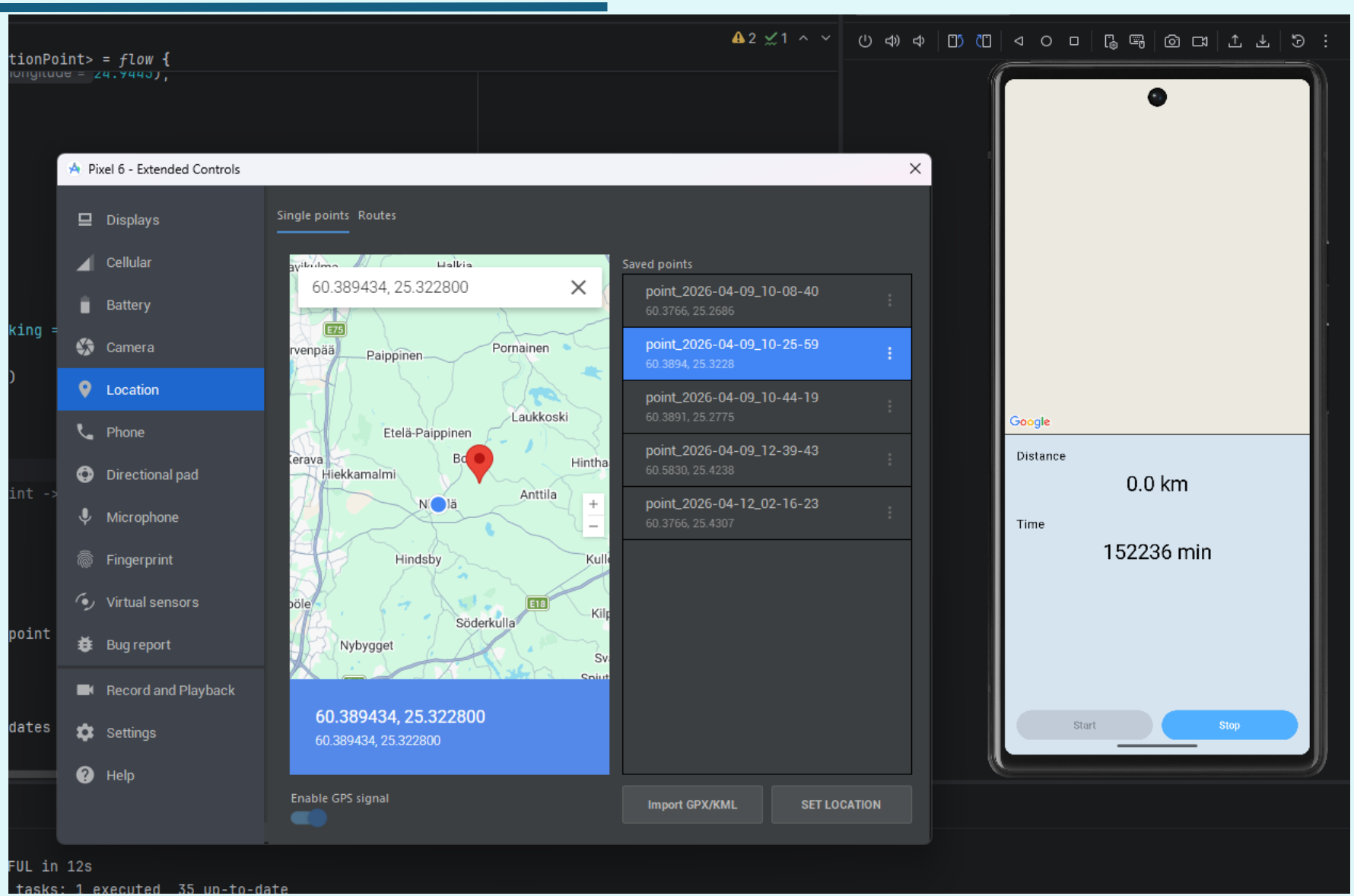
Figma:



https://www.figma.com/design

What I (Farnaz) Worked On:

Fix Issues



GPS Permission Errors:

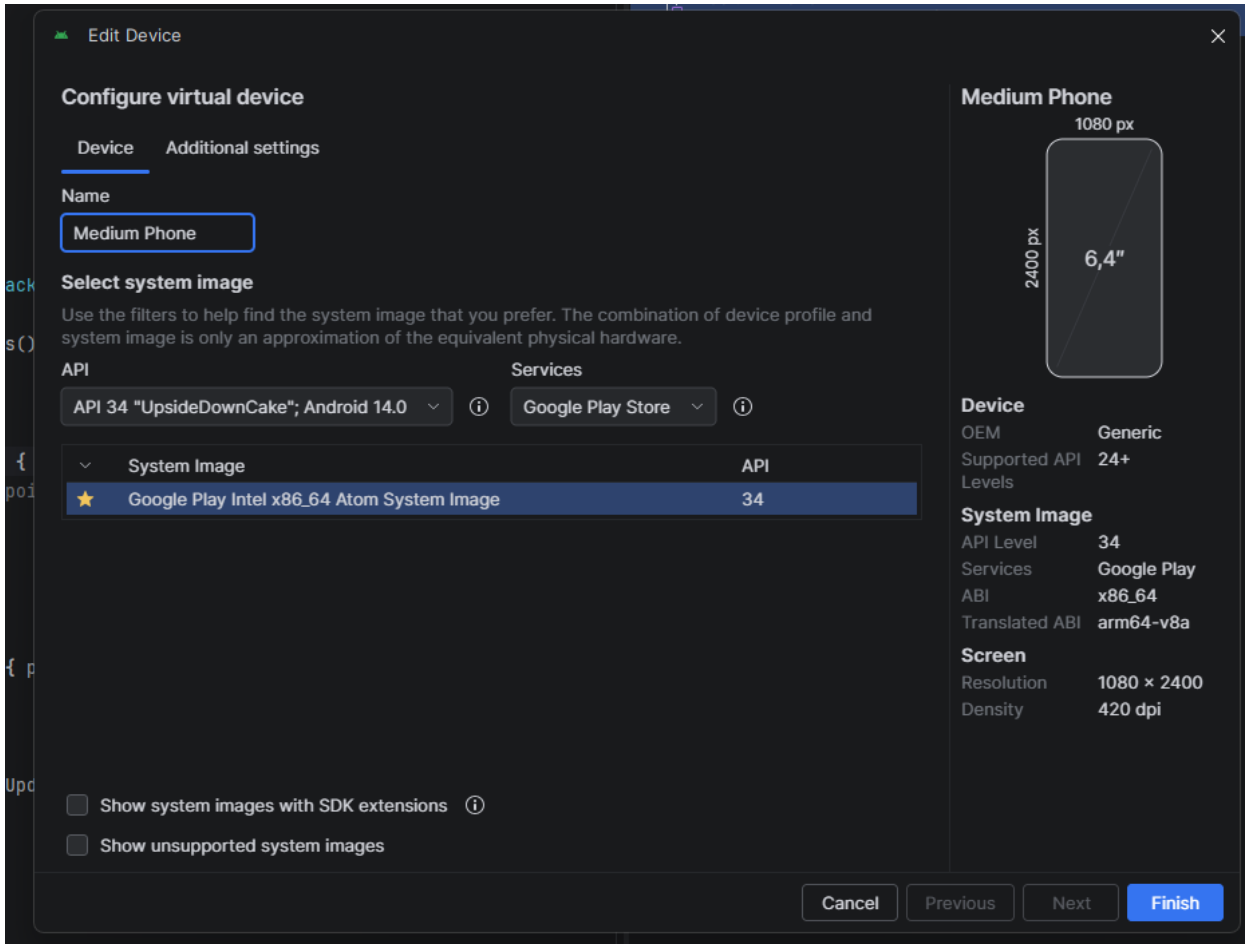
MisssingPermission

```
1 Usage
@SuppressLint( ...value = "MissingPermission")
fun startLocationUpdates(onUpdate: (LocationPoint) -> Unit) {

    if (!hasLocationPermission()) return

    setupLocationRequest()

    locationCallback = object : LocationCallback() {
        override fun onLocationResult(locationResult: LocationResult) {
            val loc: Location = locationResult.lastLocation ?: return
            onUpdate(
                LocationPoint(
                    latitude = loc.latitude,
                    longitude = loc.longitude,
                    time = loc.time
                )
```

SecurityException

```
try {
    fusedLocationProviderClient.lastLocation
        .addOnSuccessListener { loc: Location? ->
            if (loc == null) {
                onResult(null)
            } else {
                onResult(
                    LocationPoint(
                        latitude = loc.latitude,
                        longitude = loc.longitude,
                        time = loc.time
                    )
                )
            }
        }
        .addOnFailureListener {
            onResult(null)
        }

} catch (e: SecurityException) {
    e.printStackTrace()
    onResult(null)
}
```

Set emulator:

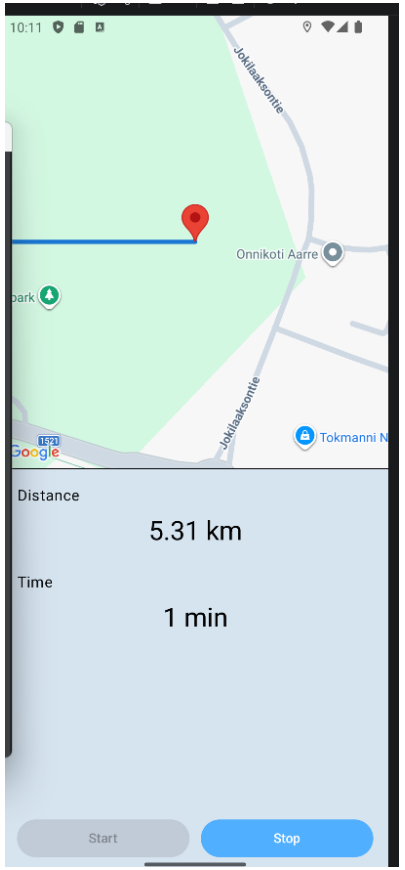


Emulator limitations:

accepts ALL points, even bad ones

val addedDistance = calculateDistance(lastPoint, point)

```
2 Usages
private fun handleNewPoint(point: LocationPoint) {

    if (lastPoint != null) {

        val distanceKm = calculateDistance( a = lastPoint!!, b = point)
        val timeDiffMs = point.time - lastPoint!!.time

        val timeDiffSec = if (timeDiffMs <= 0) 1.0 else timeDiffMs / 1000.0

        val speed = distanceKm / (timeDiffSec / 3600.0) // km/h
        // dog cannot go > 20 km/h
        // Ignore impossible speeds
        if (speed > 25) return

        // Ignore tiny GPS noise (< 1 meter)
        if (distanceKm < 0.001) return
    }

    val addedDistance = if (lastPoint != null) {
        calculateDistance( a = lastPoint!!, b = point)
    } else 0.0
```

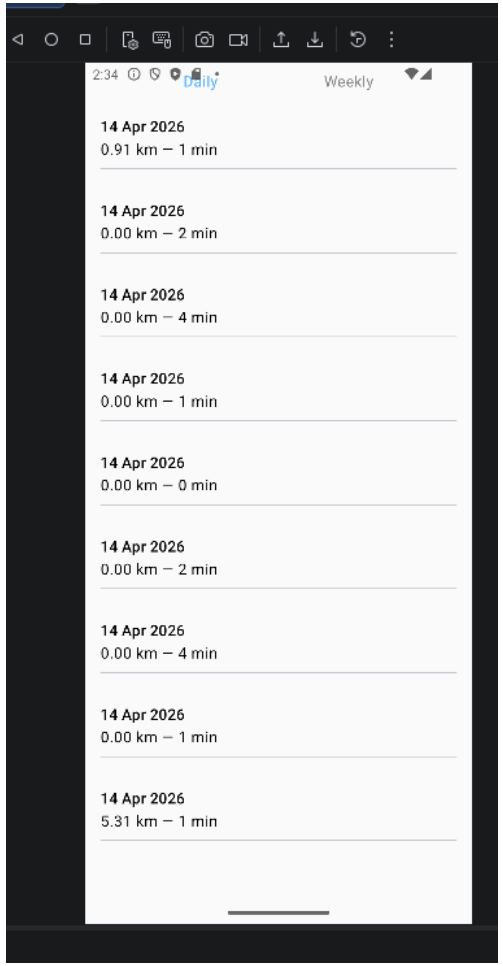


Dily/Weekly filtering:

cal.get(Calendar.WEEK_OF_YEAR) == todayWeek

```
3 Usages
private fun filterDaily(list: List<WalkUiModel>): List<WalkUiModel> {
    val cal = Calendar.getInstance()
    val todayYear = cal.get(Calendar.YEAR)
    val todayDay = cal.get(Calendar.DAY_OF_YEAR)

    return list.filter { walk ->
        cal.timeInMillis = walk.startTime
        cal.get(Calendar.YEAR) == todayYear &&
                cal.get(Calendar.DAY_OF_YEAR) == todayDay
    }
}
```

```
3 Usages
private fun filterWeekly(list: List<WalkUiModel>): List<WalkUiModel> {
    val now = System.currentTimeMillis()
    val oneWeekAgo = now - (7 * 24 * 60 * 60 * 1000L)

    return list.filter { walk ->
        walk.startTime in oneWeekAgo ≤ .. ≤ now
    }
}
}
```



Daily/weekly filtering:

allWalks = uiList
applyFilter(_uiState.value.filter)

```
private fun loadHistory() {
    viewModelScope.launch {
        repository.getAllWalks()
            .map { list -> list.map { it.toUiModel() } }
            .collect { uiList ->

                allWalks = uiList

                val currentFilter = _uiState.value.filter

                val filtered = when (currentFilter) {
                    WalkFilter.Daily -> filterDaily( list = allWalks)
                    WalkFilter.Weekly -> filterWeekly( list = allWalks)
                }

                _uiState.value = _uiState.value.copy(
                    walks = filtered
                )
            }
    }
```

Distance and duration format:

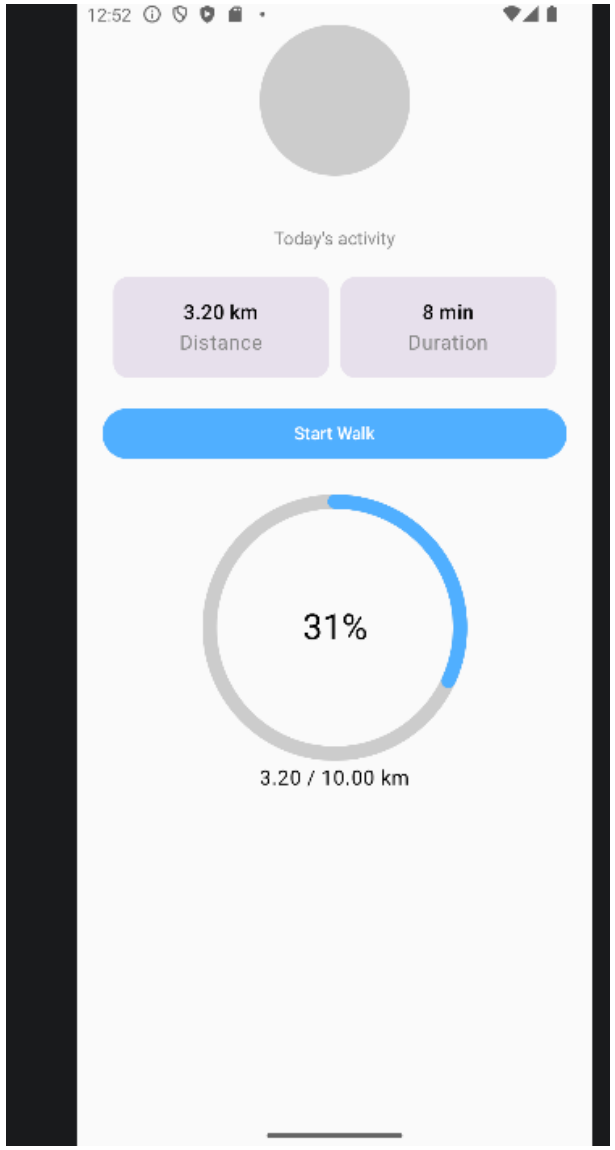
```
Text("${walk.distance} km")
Text("${walk.duration} min")
```

```
val hours = walk.timeMinutes / 60
val minutes = walk.timeMinutes % 60

val timeFormatted = if (hours > 0)
    "%d h %02d min".format( ...args = hours, minutes)
else
    "%d min".format( ...args = minutes)

Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding( all = 16.dp)
) {
    Text(walk.date, style = MaterialTheme.typography.titleMedium)
    Text("%.2f km — %s".format( ...args = walk.distanceKm, timeFormatted))
```

```
Text("${uiState.distance} km")
```

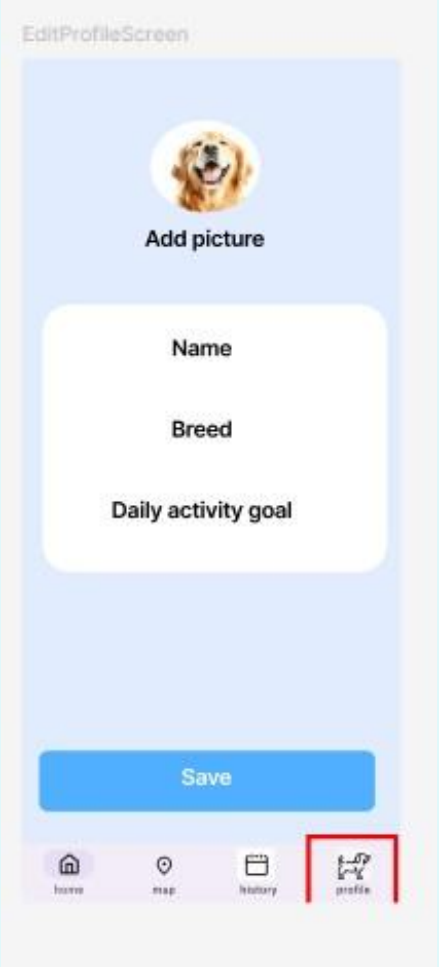
```
fun TrackingStatistics(
    uiState: TrackingUiState,
    modifier: Modifier = Modifier
) {
    val totalMinutes = uiState.time / 60000
    val hours = totalMinutes / 60
    val minutes = totalMinutes % 60

    val formattedTime = if (hours > 0)
        "%d h %02d min".format( ...args = hours, minutes)
    else
        "%d min".format( ...args = minutes)

    Column(modifier = modifier.fillMaxWidth()) {

        Text("Distance")
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = 8.dp),
            contentAlignment = Alignment.Center
        ) {
            Text(
                "%.2f km".format( ...args = uiState.distance),
```

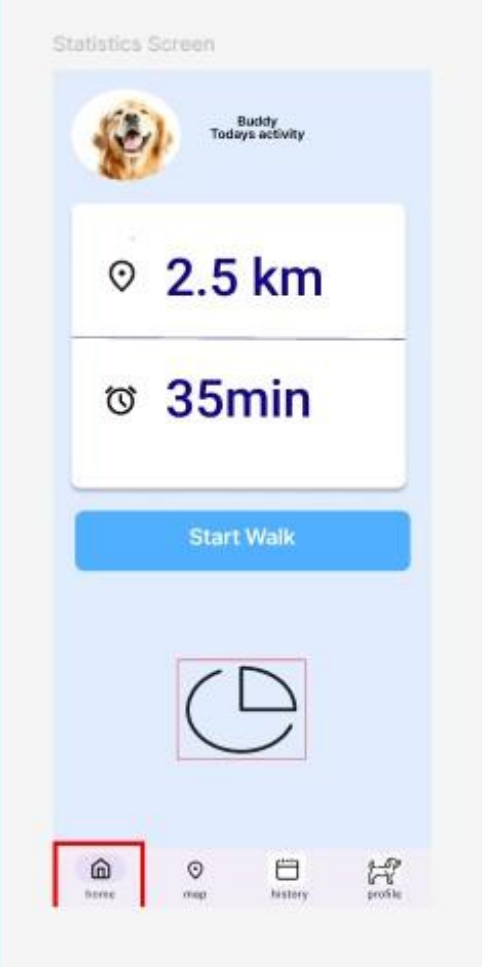
Chart:


12:52
Today's activity
3.20 km
Distance
8 min
Duration
Start Walk
31%
3.20 / 10.00 km

Emmi tasks:

- Save goals to database
- Sum today’s/week's walks: totalDistanceKm, TotalDurationMs
- Testing: Unit tests, UI Test, database tests, integration tests





Save goals to database – DogProfile

```
@Entity(tableName = "dog_profile")
data class DogProfileEntity(
    @PrimaryKey val id: Int = 0,
    val imageUri: String = "",
    val name: String = "",
    val breed: String = "",

    val dailyDistanceGoal: Float = 0.0f,
    val dailyDurationGoal: Long = 0L,

    val weeklyDistanceGoal: Float = 0.0f,
    val weeklyDurationGoal: Long = 0L
)
```

*Default values

```
@Dao
interface DogProfileDao {

    1 Usage
    @Query( value = "SELECT * FROM dog_profile WHERE id = 0")
    fun getProfile(): Flow<DogProfileEntity?>

    1 Usage
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertProfile(profile: DogProfileEntity)
}
```

```
6 Usages   2 Implementations
interface DogProfileRepository {
    3 Usages   2 Implementations
    fun getProfile(): Flow<DogProfileEntity?>
    1 Usage   2 Implementations
    suspend fun saveProfile(profile: DogProfileEntity)
}
```

```
class DogProfileRepositoryImpl(
    private val dao: DogProfileDao
) : DogProfileRepository {

    2 Usages
    override fun getProfile() = dao.getProfile()

    1 Usage
    override suspend fun saveProfile(profile: DogProfileEntity) {
        dao.upsertProfile(profile)
    }
}
```

*Upsert + Replace

```kotlin
@Composable
fun ProfileScreen(
    viewModel: ProfileViewModel
) {
    val state by viewModel.uiState.collectAsState()

    Column(modifier = Modifier.padding( all = 16.dp)) {

        Text("Dog Profile")

        TextField(
            value = state.name,
            onValueChange = viewModel::onNameChange,
            label = { Text("Name") }
        )

        TextField(
            value = state.breed,
            onValueChange = viewModel::onBreedChange,
            label = { Text("Breed") }
        )

        TextField(
            value = state.dailyDistanceGoal,
            onValueChange = viewModel::onDailyDistanceChange,
            label = { Text("Daily distance") }
        )

        TextField(
            value = state.dailyDurationGoal,
            onValueChange = viewModel::onDailyDurationChange,
            label = { Text("Daily duration") }
        )

        TextField(
            value = state.weeklyDistanceGoal,
            onValueChange = viewModel::onWeeklyDistanceChange,
            label = { Text("Weekly distance") }
        )

        TextField(
```

```kotlin
data class ProfileUiState(
    val imageUri: String = "",
    val name: String = "",
    val breed: String = "",
    val dailyDistanceGoal: String = "",
    val dailyDurationGoal: String = "",

    val weeklyDistanceGoal: String = "",
    val weeklyDurationGoal: String = "",
    val isSaved: Boolean = false
)
```

*User input is as Strings to allow real-time editing in text fields without errors

```
class ProfileViewModel(
    private val repository: DogProfileRepositoryImpl
) : ViewModel() {

    // Constant stream of profile data from the database to keep the UI screens synced
    val dogProfile = repository.getProfile()
        .stateIn(viewModelScope, started = SharingStarted.WhileSubscribed( stopTimeoutMillis = 5000), initialValue = null)

    // Temporary state to hold user input before it is saved to the database
    9 Usages
    private val _uiState = MutableStateFlow( value = ProfileUiState())
    1 Usage
    val uiState: StateFlow<ProfileUiState> = _uiState.asStateFlow()

    init {
        loadInitialData()
    }

    // Fetches the existing profile once to fill the input fields when the screen opens
    1 Usage
    private fun loadInitialData() {
        viewModelScope.launch {

            val initialProfile = repository.getProfile().filterNotNull().firstOrNull()

            initialProfile?.let { dogProfile ->
                _uiState.update {
                    it.copy(
                        name = dogProfile.name,
                        breed = dogProfile.breed,
                        dailyDistanceGoal = dogProfile.dailyDistanceGoal.toString(),
                        dailyDurationGoal = dogProfile.dailyDurationGoal.toString(),
                        weeklyDistanceGoal = dogProfile.weeklyDistanceGoal.toString(),
                        weeklyDurationGoal = dogProfile.weeklyDurationGoal.toString(),
                        imageUri = dogProfile.imageUri
                    )
                }
```

```
    // Updates functions to sync text field changes with the temporary UI state
    1 Usage
    fun onNameChange(value: String) = _uiState.update { it.copy(name = value) }
    1 Usage
    fun onBreedChange(value: String) = _uiState.update { it.copy(breed = value) }
    1 Usage
    fun onDailyDistanceChange(value: String) = _uiState.update { it.copy(dailyDistanceGoal = value) }
    1 Usage
    fun onDailyDurationChange(value: String) = _uiState.update { it.copy(dailyDurationGoal = value) }
    1 Usage
    fun onWeeklyDistanceChange(value: String) = _uiState.update { it.copy(weeklyDistanceGoal = value) }
    1 Usage
    fun onWeeklyDurationChange(value: String) = _uiState.update { it.copy(weeklyDurationGoal = value) }


    // Converts the temporary state back to an entity and saves it to the database
    1 Usage
    fun save() {
        viewModelScope.launch {
            val state = _uiState.value

            val entity = DogProfileEntity(
                id = 0,
                imageUri = "testImage",
                name = state.name,
                breed = state.breed,
                // Converts String inputs to correct numeric types defaulting to 0 if invalid
                dailyDistanceGoal = state.dailyDistanceGoal.toFloatOrNull() ?: 0f,
                dailyDurationGoal = state.dailyDurationGoal.toLongOrNull() ?: 0L,
                weeklyDistanceGoal = state.weeklyDistanceGoal.toFloatOrNull() ?: 0f,
                weeklyDurationGoal = state.weeklyDurationGoal.toLongOrNull() ?: 0L
            )

            repository.saveProfile(entity)
        }
    }
```

Sum today/week walks – Statistics data layer

WalkDao:

```
//Statistics
2 Usages
@Query( value = """SELECT SUM(distance) FROM walks WHERE startTime >= :startOfDay""")
fun getTotalDistanceSince(startOfDay: Long): Flow<Float?>

2 Usages
@Query( value = """SELECT SUM(duration) FROM walks WHERE startTime >= :startOfWeek""")
fun getTotalDurationSince(startOfWeek: Long): Flow<Long?>
```

+ StatisticsViewModel - base

WalkRepository:

```
// Statistics
1 Usage  2 Implementations
fun getTodayDistance(): Flow<Float?>
1 Usage  2 Implementations
fun getTodayDuration(): Flow<Long?>
1 Usage  2 Implementations
fun getWeekDistance(): Flow<Float?>
1 Usage  2 Implementations
fun getWeekDuration(): Flow<Long?>
```

WalkRepositoryImpl:

```
// Statistics
override fun getTodayDistance() =
    dao.getTotalDistanceSince( startOfDay = TimeUtils.getStartOfDay())

override fun getTodayDuration() =
    dao.getTotalDurationSince( startOfWeek = TimeUtils.getStartOfDay())

override fun getWeekDistance() =
    dao.getTotalDistanceSince( startOfDay = TimeUtils.getStartOfWeek())

override fun getWeekDuration() =
    dao.getTotalDurationSince( startOfWeek = TimeUtils.getStartOfWeek())
```

Testing – In progress

- **Problems:**

We had initially only one concrete repository class (WalkRepository, DogProfileRepositpry, GPSRepository). ViewModel depended directly on the implementation and unit testing was difficult. -> refactored repository into interface + implementation.

Questions

- UseCase/domain layer?
- Refactoring WalkRepository?

Sprint 4 goals and backlog:

- Improve icon consistency (size, color, alignment).
- Improve usability and clarity across all screens.
- Polish navigation flow and ensure consistent bottom-nav behavior.
- Enhance visual hierarchy so key actions stand out (Start Walk, Save, Edit Profile).
- Refine layout responsiveness for different screen sizes.
- Prepare UI for final presentation/demo.
- Create navigation bar
- Align fields with consistent spacing.
- Increase contrast for text and icons.
- Ensure all screens visually match the design system.